

Práctica 4 PDO | David Rodríguez Aparicio

En esta práctica hemos creado algunas extensiones de las clases que ya teníamos mediante herencia, y creando una clase abstracta, que en este caso es Sorpresa.

Además hemos añadido la herencia de Jugador, JugadorEspeculador, que tiene ciertas ventajas sobre los normales, además se ha añadido su respectiva carta de Sorpresa.

En principio se mantienen todas las funcionalidades como anteriormente, simplemente hemos hecho una refactorización del código y añadido un par de mecánicas.

Herencias de Casilla

```
package civitas;

public class CasillaCalle extends Casilla {

    TituloPropiedad tituloPropiedad;

    /* ----- Constructores ----- */

    CasillaCalle(TituloPropiedad titulo){

        super(titulo.getNombre());
        this.tituloPropiedad = titulo;

    }

    /* ----- MODULOS ----- */
    @Override
    void recibeJugador(int iActual, Jugador[] Jugadores){
        if (jugadorCorrecto(iActual, Jugadores)) {
            informe(iActual, Jugadores);

            // System.out.println("\n recibirjugador_calle - tiene propietario" +
            tituloPropiedad.tienePropietario());

            if (!tituloPropiedad.tienePropietario() /* && this.tipo !=
            TipoCasilla.CALLE */ ) {
                Jugadores[iActual].puedeComprarCasilla();
                // Diario.getInstance().ocurreEvento("El jugador " +
                Jugadores[iActual].getNombre() + " ha recibido: " +
                this.tituloPropiedad.getNombre() );
            }
        }
    }
}
```

```
        } else {

            // System.out.println("\n Entada a recibir calle -
tramitarAlquiler \n");
            tituloPropiedad.tramitarAlquiler(Jugadores[iActual]);

        }
    }
}

@Override
public String toString(){
    String info = "Casilla{" +
        "nombre='" + this.getNombre() + '\'' + ", titulo=" +
        tituloPropiedad.getNombre() + "}";
    return info;
}

TituloPropiedad getTituloPropiedad(){
    return this.tituloPropiedad;
}
}
```

```
package civitas;

public class CasillaImpuesto extends Casilla {

    private float importe;

    /* ----- Constructores ----- */

    CasillaImpuesto(float cantidad,String nombre){

        this.importe = cantidad;
        super(nombre);
    }

    /* ----- MODULOS ----- */

    @Override
    void recibeJugador(int iActual, Jugador[] Jugadores){
        if (jugadorCorrecto(iActual, Jugadores)) {
            informe(iActual, Jugadores);
        }
    }
}
```

```
        Diario.getInstance().ocurreEvento("El jugador " +
Jugadores[iActual].getNombre() + " ha caído en: " + this.getNombre() );

        Jugadores[iActual].pagaImpuesto(importe);
    }
}

@Override
public String toString(){
    String info = "Casilla{" +
        "nombre='" + this.getNombre() + '\'' + ", importe= " +
this.importe + "}";
    return info;
}
}
```

```
package civitas;
```

```
public class CasillaJuez extends Casilla{
```

```
    private int carcel;
```

```
    /* ----- Constructores ----- */
```

```
    CasillaJuez(int numCasillaCarcel, String nombre){
```

```
        super(nombre);
```

```
        this.carcel = numCasillaCarcel;
```

```
    }
```

```
    /* ----- MODULOS ----- */
```

```
    @Override
```

```
    void recibeJugador(int iActual, Jugador[] Jugadores){
```

```
        if (jugadorCorrecto(iActual, Jugadores)) {
```

```
            informe(iActual, Jugadores);
```

```
            Jugadores[iActual].encarcelar(carcel);
```

```
        }
```

```
    }
```

```
    @Override
```

```
    public String toString(){
```

```
        String info = "Casilla{" +
```

```
            "nombre='" + this.getNombre() + '\'' + ", carcel=" +
```

```
this.carcel + "}";
```

```
        return info;
    }
}

package civitas;

public class CasillaSorpresa extends Casilla {

    Sorpresa sorpresa;
    MazoSorpresa mazo;

    /* ----- Constructores ----- */

    CasillaSorpresa(MazoSorpresa mazo, String nombre){

        super(nombre);
        this.mazo = mazo;
    }

    /* ----- MODULOS ----- */

    @Override
    void recibeJugador(int iActual, Jugador[] Jugadores){
        if (jugadorCorrecto(iActual, Jugadores)) {
            informe(iActual, Jugadores);
            this.sorpresa = this.mazo.siguiente();
            this.sorpresa.aplicarAJugador(iActual, Jugadores);
        }
    }

    @Override
    public String toString(){
        String info = "Casilla{" +
            "nombre='" + this.getNombre() + '\'' + ", mazoSorpresas" +
        "}";
        return info;
    }
}
```

Herencias de Sorpresa

```
package civitas;

public abstract class Sorpresa{
    private String texto;
    protected MazoSorpresa mazo;

    /* ----- CONSTRUCTORES ----- */

    /* Constructor para herencias */
    Sorpresa(){

    }

    Sorpresa(String texto) {
        this.texto = texto;
        this.mazo = null;
    }

    /* ----- METODOS ----- */

    abstract void aplicarAJugador(int actual, Jugador[] jugadores);

    protected void informe(int actual, Jugador[] Jugadores){
        Diario.getInstance().ocurreEvento( "Se le está aplicando una sorpresa a: "
+ Jugadores[actual] );
    }

    public boolean jugadorCorrecto(int actual, Jugador[] Jugadores){
        return (actual >= 0 && actual < Jugadores.length);
    }

    void salirDelMazo(){
        if ((this instanceof SorpresaSalirCarcel ) && mazo != null) {
            mazo.inhabilitarCartaEspecial(this);
        }
    }

    @Override
    public String toString() {
        return this.texto;
    }

    void usada(){
        if ( (this instanceof SorpresaSalirCarcel ) && mazo != null) {
            mazo.habilitarCartaEspecial(this);
        }
    }
}
```

```
}

}

package civitas;

public class SorpresaConvertir extends Sorpresa {

    private final float fianza;

    /* ----- CONSTRUCTOR ----- */
    public SorpresaConvertir(float fianza, String texto) {
        super(texto);
        this.fianza = fianza;
    }

    /* ----- APLICAR ----- */
    @Override
    void aplicarAJugador(int actual, Jugador[] jugadores) {
        if (!jugadorCorrecto(actual, jugadores)) return;

        Jugador jugador = jugadores[actual];

        // Evita convertir dos veces (si ya es especulador)
        if (jugador instanceof JugadorEspeculador) {
            Diario.getInstance().ocurreEvento("El jugador " + jugador.getNombre()
+ " ya es ESPECULADOR.");
            return;
        }

        JugadorEspeculador nuevo = new JugadorEspeculador(jugador, fianza);

        // Sustitución en el array (esto ES la conversión real)
        jugadores[actual] = nuevo;

        Diario.getInstance().ocurreEvento(
            "El jugador " + jugador.getNombre() + " se convierte en ESPECULADOR
con fianza " + fianza
        );
    }
}
```

```
package civitas;

public class SorpresaIrCarcel extends Sorpresa {

    Tablero tablero;

    /* ----- CONSTRUCTORES ----- */

    /* Para IRCASILLA */
    SorpresaIrCarcel(Tablero tablero,String texto) {

        this.tablero = tablero;

        super(texto);

    }

    @Override
    public void aplicarAJugador(int actual, Jugador[] Jugadores){
        if(jugadorCorrecto(actual, Jugadores)){
            Jugadores[actual].encarcelar( tablero.getCarcel() );/* Casilla de la
carcel */
            Diario.getInstance().ocurreEvento("Se ha encarcelado al jugador: " +
Jugadores[actual].getNombre() );
        }
    }
}
```

```
package civitas;

public class SorpresaIrCasilla extends Sorpresa{

    Tablero tablero;
    private int valor;

    /* ----- CONSTRUCTORES ----- */

    /* Para IRCASILLA */
    SorpresaIrCasilla(Tablero tablero,String texto,int valor) {

        this.tablero = tablero;
        this.valor = valor;

        super(texto);

    }
}
```

```

@Override
public void aplicarAJugador(int actual, Jugador[] Jugadores){
    if(jugadorCorrecto(actual, Jugadores)){
        informe(actual, Jugadores);
        Diario.getInstance().ocurreEvento("Se ha usado irCasilla en: " +
Jugadores[actual].getNombre() );

        int casillaActual = Jugadores[actual].getNumCasillaActual();

        int tirada = tablero.calcularTirada(casillaActual, this.valor); /*
Falta añadir el numero del dado. ?¿ */

        int nuevaPosicion = tablero.nuevaPosicion(casillaActual, tirada);

        Jugadores[actual].moverACasilla(nuevaPosicion);

        tablero.getCasilla(nuevaPosicion).recibeJugador(actual, Jugadores);
    }
}
}

```

```
package civitas;
```

```

public class SorpresaPagarCobrar extends Sorpresa {

    private int valor;

    /* ----- CONSTRUCTORES ----- */

    SorpresaPagarCobrar(int valor, String texto) {

        this.valor = valor;
        super(texto);

    }

    @Override
    public void aplicarAJugador(int actual, Jugador[] Jugadores){
        if(jugadorCorrecto(actual, Jugadores)){
            informe(actual, Jugadores);
            Diario.getInstance().ocurreEvento("Se ha usado pagarCobrar en: " +
Jugadores[actual].getNombre() );
            Jugadores[actual].modificarSaldo(this.valor);

        }
    }
}

```

```
}
```

```
package civitas;

public class SorpresaPorCasaHotel extends Sorpresa{

    private int valor;

    /* ----- CONSTRUCTORES ----- */

    SorpresaPorCasaHotel(int valor, String texto) {
        this.valor = valor;

        super(texto);
    }

    @Override
    public void aplicarAJugador(int actual, Jugador[] Jugadores){

        if (jugadorCorrecto(actual, Jugadores)) {
            informe(actual, Jugadores);
            Diario.getInstance().ocurreEvento("Se ha usado porCasaHotel en: " +
Jugadores[actual].getNombre() );

            int numCasasHoteles = Jugadores[actual].cantidadCasasHoteles();

            int cantidad = valor * numCasasHoteles;

            Jugadores[actual].modificarSaldo(cantidad);
        }

    }

}
```

```
package civitas;

public class SorpresaPorJugador extends Sorpresa {

    private int valor;
```

```
/* ----- CONSTRUCTORES ----- */

SorpresaPorJugador( int valor, String texto) {

    this.valor = valor;

    super(texto);

}

@Override
public void aplicarAJugador(int actual, Jugador[] Jugadores){

    if (jugadorCorrecto(actual, Jugadores)) {

        informe(actual, Jugadores);
        Diario.getInstance().ocurreEvento("Se ha usado porJugador en: " +
Jugadores[actual].getNombre() );

        int numJugadores = Jugadores.length;

        SorpresaPagarCobrar paga = new SorpresaPagarCobrar( -valor, "Paga al
jugador " + Jugadores[actual].getNombre());

        for (int i = 0; i < numJugadores; i++) {
            if ( i != actual ) {
                paga.aplicarAJugador(actual, Jugadores);
            }
        }

        SorpresaPagarCobrar cobra = new SorpresaPagarCobrar(valor *
(numJugadores - 1), "Cobra del resto de Jugadores");

        cobra.aplicarAJugador(actual, Jugadores);
    }
}

}

package civitas;

public class SorpresaSalirCarcel extends Sorpresa{

    /* ----- CONSTRUCTORES ----- */

    SorpresaSalirCarcel(MazoSorpresa mazo, String texto) {
```

```

        super(texto);
    }

    @Override
    public void aplicarAJugador(int actual, Jugador[] Jugadores){
        if (jugadorCorrecto(actual, Jugadores)) {

            informe(actual, Jugadores);
            Diario.getInstance().ocurreEvento("Se ha usado salirCarcel en: " +
Jugadores[actual].getNombre() );

            boolean alguienTieneSalvoconducto = false;

            for (int i = 0; i < Jugadores.length && !alguienTieneSalvoconducto;
i++) {
                if (Jugadores[i].tieneSalvoconducto()) {
                    alguienTieneSalvoconducto = true;
                }
            }

            if (!alguienTieneSalvoconducto) {
                Jugadores[actual].obtenerSalvoconducto(this);
                salirDelMazo();
            }
        }
    }
}

```

Herencia de Jugador, Especulador

```

package civitas;

import java.util.ArrayList;

public class JugadorEspeculador extends Jugador {

    private static final int FACTOR_ESPECULADOR = 2;
    private float fianza;

    /* ----- CONSTRUCTOR ----- */

    public JugadorEspeculador(Jugador otro, float fianza) {
        super(otro);
        this.fianza = fianza;
    }
}

```

```
        for (TituloPropiedad propiedad : this.propiedades) {
            propiedad.setPropietario(this);
        }

        Diario.getInstance().ocurreEvento(
            "El jugador " + this.getNombre() + " se ha convertido en ESPECULADOR
(fianza: " + fianza + ")
        );
    }

    /* ----- METODOS ----- */

    @Override
    boolean pagaImpuesto(float cantidad) {
        // El especulador paga la mitad
        return super.pagaImpuesto(cantidad / FACTOR_ESPECULADOR);
    }

    @Override
    protected boolean debeSerEncarcelado() {

        // Si ya tiene salvoconducto - comportamiento normal
        if (!super.debeSerEncarcelado()) {
            return false;
        }

        // Si puede pagar la fianza, no entra en la cárcel
        if (getSaldo() >= fianza) {
            paga(fianza);
            Diario.getInstance().ocurreEvento(
                "El jugador " + getNombre() + " paga la fianza (" + fianza + ") y
evita la cárcel"
            );
            return false;
        }

        return true;
    }

    @Override
    public int getCasasMax() {
        return super.getCasasMax() * FACTOR_ESPECULADOR;
    }

    @Override
    public int getHotelesMax() {
        return super.getHotelesMax() * FACTOR_ESPECULADOR;
    }

    @Override
    public String toString() {
        return super.toString() + " [ESPECULADOR | fianza=" + fianza + "];
    }
```

```
}  
}
```